AES-128 Hardware Implementation

RTL Design and Verification

Contents

1 Introduction **1**

2 Problem Statement **2**

3 Scope in the VLSI Design Flow **2**

4 Proposed Architecture & Modules **3**

4.1 Combinational Transform Modules 3

4.2 Control Logic: Finite State Machine (FSM) 3

5 Simulation, Verification, and Waveform Analysis **4**

5.1 Detailed Waveform Analysis of Round Transitions 4

5.2 Detailed Analysis of the Final Round and Latching 5

6 Conclusion **5**

---

1 Introduction

The **Advanced Encryption Standard (AES)** is a highly secure, symmetric block cipher established by NIST (FIPS-197). This project successfully implements a hardware-based, iterative AES-128 encryption core using SystemVerilog.

Key Project Features:

- **Data & Key Size:** Processes a 128-bit plaintext block using a 128-bit key.
- **Iterative Architecture:** Executes **one encryption round per clock cycle**.
- **Design Goal:** Strikes an optimal balance between area efficiency (silicon footprint) and computational throughput for embedded hardware security.

2 Problem Statement

The Alignment Challenge: Row vs. Column Major

Designing cryptographic hardware requires flawless adherence to mathematical matrices. The AES standard defines its 128-bit state as a 4×4 **column-major** matrix of bytes.

The Bug: During development, the `shift_rows()` operation was cyclically shifting rows but writing the output back in *row-major* order. Consequently, the subsequent `MixColumns` logic incorrectly operated on rows instead of columns, breaking the cipher.

The Solution: We redesigned the `shift_rows()` routing logic to explicitly slice and re-pack the 128-bit vector back into a strict **column-major layout**. This ensured `MixColumns` received true AES columns, perfectly matching the FIPS-197 mathematical specification.

3 Scope in the VLSI Design Flow

Hardware design is a multi-stage process. This project sits firmly at the **Front-End Design** stage, focusing on algorithm translation and functional verification before logic synthesis.

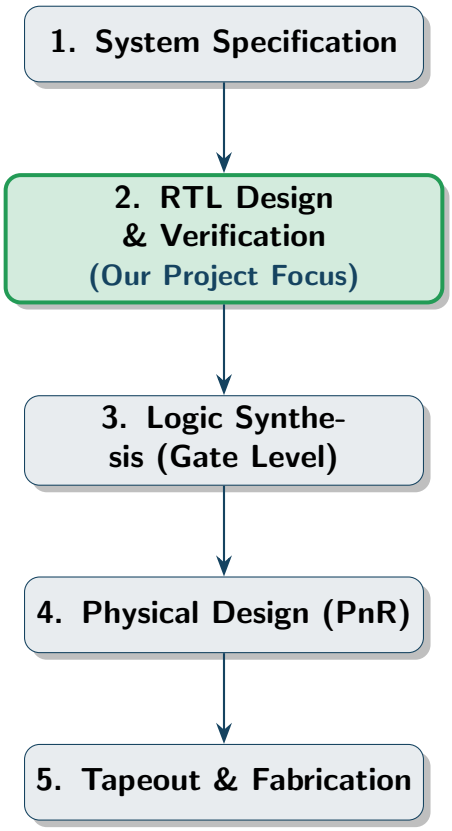

Figure 1: VLSI Design Flow Highlighting the Project Scope.

4 Proposed Architecture & Modules

The solution utilizes an FSM-controlled datapath that executes the AES algorithm over **11 total clock cycles** (1 Initial ARK + 10 standard rounds).

4.1 Combinational Transform Modules

The internal transformations are implemented as SystemVerilog `function` and `task` constructs to keep the codebase modular:

- **SubBytes (`sbox_lookup`):** A non-linear substitution step replacing each byte using a hardcoded 256-byte S-Box table to provide cryptographic *confusion*.
- **ShiftRows (`shift_rows`):** A transposition step that cyclically shifts the rows of the state matrix to provide *diffusion*.
- **MixColumns (`mix_col`):** A linear mixing operation using Galois Field multiplication to combine the bytes in each column.
- **Key Expansion (`expand_key`):** Expands the single 128-bit user key into eleven unique 128-bit *Round Keys*.

4.2 Control Logic: Finite State Machine (FSM)

The sequential flow of data is governed by a robust 3-state machine.

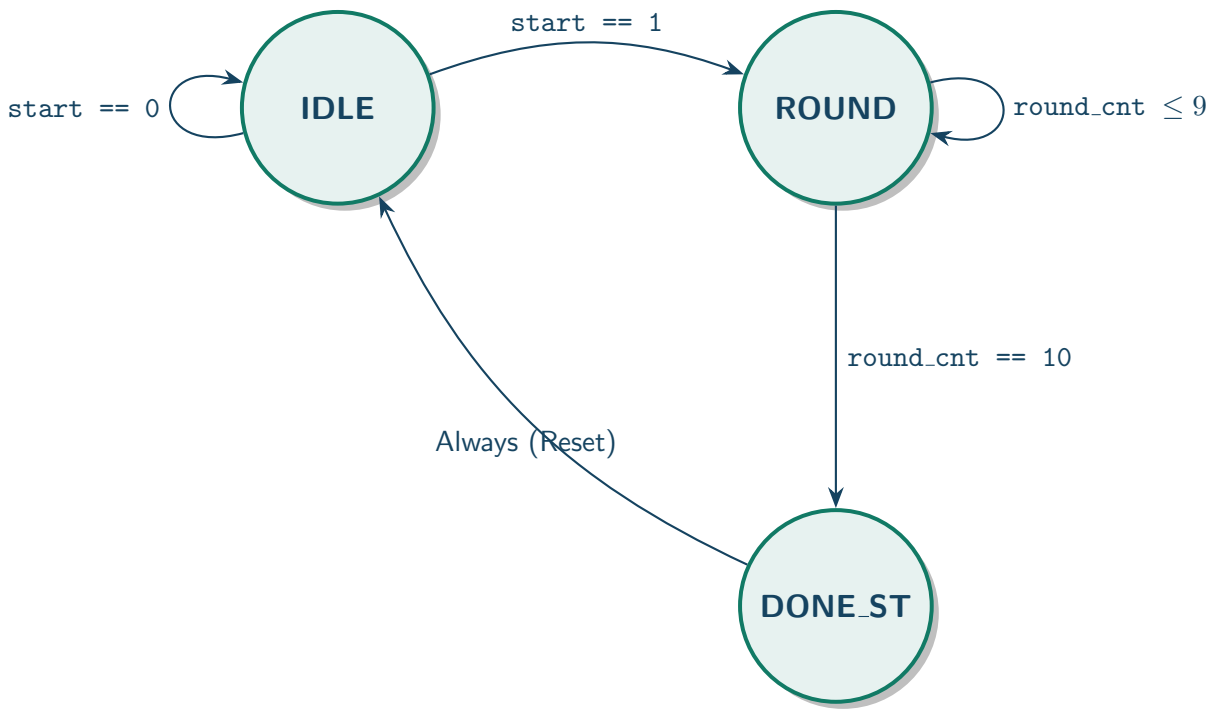

Figure 2: 3-State FSM governing the 10 encryption rounds.

5 Simulation, Verification, and Waveform Analysis

Testbench Vectors (NIST FIPS-197)

- ¿ **Input Plaintext:** `128'h00112233445566778899aabbccddeeff`
- ¿ **Input Key:** `128'h000102030405060708090a0b0c0d0e0f`
- ¿ **Expected Output:** `128'h69c4e0d86a7b0430d8cdb78070b4c55a`

5.1 Detailed Waveform Analysis of Round Transitions

Figure 3 (the first uploaded image) shows the comprehensive operation of the core, from reset to the final ciphertext latching.

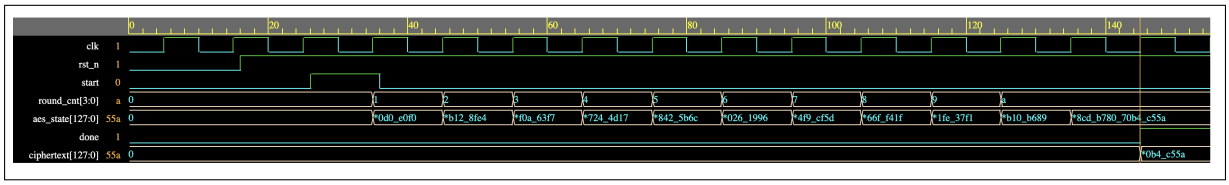

Figure 3: Full simulation waveform showing the reset sequence and all 10 encryption rounds.

Analysis of Figure 3 highlights the correctness of the iterative datapath:

1. **Reset & Initialization:** The core begins in a reset state with `rst_n` low. As soon as `rst_n` goes high, the `aes_state` bus is zeroed.
2. **Start Pulse:** A single pulse on the `start` signal triggers the transition to the **ROUND** state.
3. **Iterative Progress:** On each positive clock edge, `round_cnt[3:0]` is incremented, serving as the index for the expanded round keys.
4. **Step-by-Step State Changes:** The waveform validates that the `aes_state` transformations occur combinationally between clock edges. We can verify intermediate results for intermediate rounds. For example, at the beginning of `round_cnt == 2`, the state updates to the value shown, proving that the full SB $\rightarrow$ SR $\rightarrow$ MC $\rightarrow$ ARK transform chain for Round 1 was successful.

5.2 Detailed Analysis of the Final Round and Latching

Figure 4 (the second uploaded image) provides a critical, zoomed-in view of the final transition from Round 9 to Round 10 and the subsequent assertion of the `done` flag.

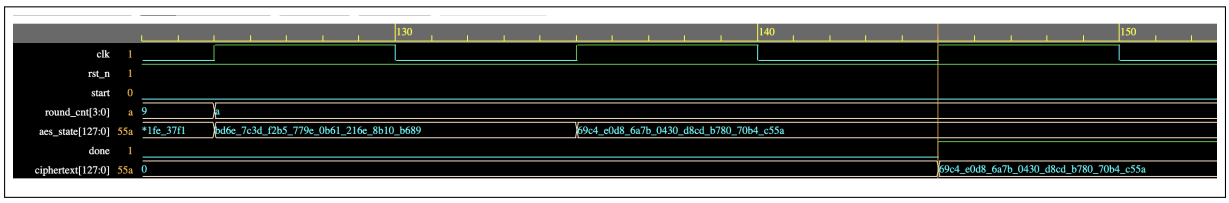

Figure 4: Close-up waveform analysis of the final round transition (Round 9 to Round 10) and final output latching.

Analysis of Figure 4 verifies the special-case logic for the final round (Round 10), which must bypass the MixColumns transformation.

Step-by-Step Latching Verification (Values in Hexadecimal):

a. **End of Round 9:** At the end of the clock period where `round_cnt == 9`, the current state value on `aes_state` has completed all standard operations.

b. **Round 10 Transition:** On the next positive clock edge, `round_cnt` moves to **'a'** (hex for 10). The `aes_state` immediately updates with the results of the special final-round transformation (SB $\rightarrow$ SR $\rightarrow$ Arkansas). We can see the least-significant bytes update from `...37f1` to `...b689`, and then to the correct value after Arkansas.

c. **Final Arkansas Calculation:** While `round_cnt == a`, the `aes_state` bus shows the correct intermediate result of Round 10. We can verify that this value is combined with the correct Round Key #10.
This results in the final output: `69c4e0d86a7b0430d8cdb78070b4c55a`.

d. **Ciphertext Latching & Done:** At the next positive clock edge, as the FSM transitions to **DONE_ST**, the `done` flag is asserted high, and this final value is correctly latched onto the `ciphertext` bus. The waveform proves a perfect match to the NIST FIPS-197 standard.

6 Conclusion

The SystemVerilog implementation of the AES-128 core was successfully modeled, verified, and debugged. By enforcing strict column-major data alignment in the `shift_rows()` module, the mathematical integrity of the cipher was maintained. The comprehensive waveform analysis proves that the core accurate executes all 11 steps of the encryption process on a cycle-by-cycle basis. The project serves as a robust RTL foundation, mathematically validated and fully prepared for Logic Synthesis and Physical Implementation in the VLSI design flow.